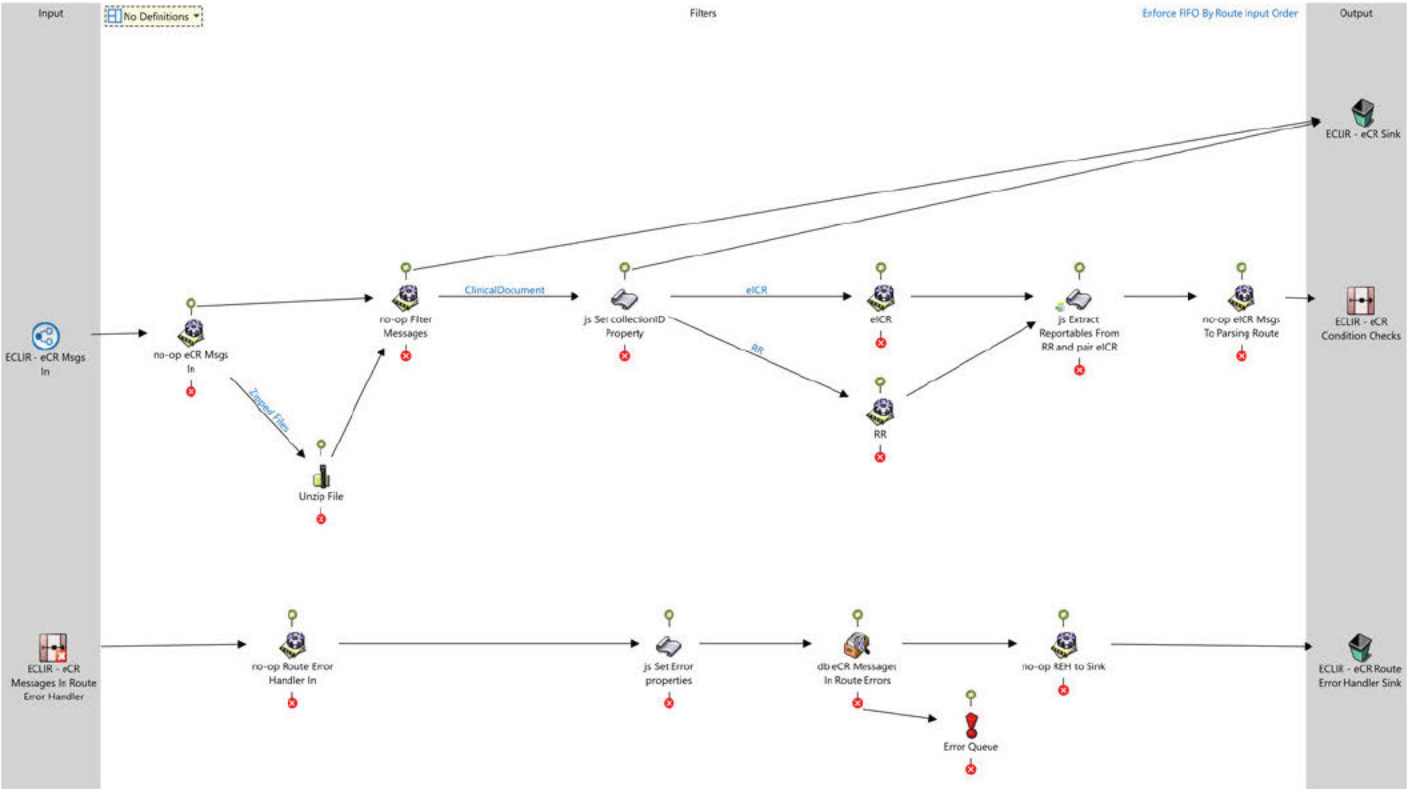




ECLIR - eCR Messages In

Route



Notes

Purpose:

This route receives RR (Reportability Response) and eICR (Initial Case Report) documents from the existing **Route eICR -RR Route - Processing**. RR and eICR documents are paired up to extract relevant reportable conditions associated with the eICR documents. Data from the RR document is parsed and stored in message properties to be written into the relevant eCR tables in the ECLIR *ECLIR - eICR Message Parsing*

Inputs

ECLIR - eCR Msgs In.....	Dynamic Router	5
--------------------------	----------------	---

Outputs

ECLIR - eCR Sink.....	Sink	12
ECLIR - eCR Condition Checks.....	Route	101
ECLIR - eCR Route Error Handler Sink.....	Sink	11

Filters

db eCR Messages In Route Errors.....	Database LookUp	46
--------------------------------------	-----------------	----

Name	Page
 eICR..... No-operation	42
 Error Queue..... Error Queue	41
 js Extract Reportables From RR and pair eICR..... JavaScript (v.1)	42
 js Set collectionID Property..... JavaScript (v.1)	39
 js Set Error properties..... JavaScript (v.1)	49
 no-op eCR Msgs In..... No-operation	49
 no-op eICR Msgs To Parsing Route..... No-operation	38
 no-op Filter Messages..... No-operation	37
 no-op REH to Sink..... No-operation	42
 no-op Route Error Handler In..... No-operation	38
 RR..... No-operation	38
 Unzip File..... Zip / Unzip	37

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



Unzip File

Zip / Unzip

On ECLIR - eCR Messages In route

Filter

Notes

Purpose: This Zip/Unzip filter unzips eCR files received from existing routes.

Configuration Properties

Mode	Unzip
On Non-zip File Receipt	Send to error connector
On Empty Zip File Receipt	Send to error connector
Filename Encoding	Default platform encoding

Destinations

Name	Page
no-op Filter Messages.....No-operation	37



no-op Filter Messages

No-operation

On ECLIR - eCR Messages In route

Filter

Notes

Purpose: This No-operation filter allows for filtering of messages in the conditional connector following to discard any messages that are not Clinical Documents.

Destinations

Name	Page
Output Condition: ClinicalDocument	
Body contains '<ClinicalDocument' ignoring case	
js Set collectionID Property..... JavaScript (v.1)	39
Error Connector	Page
ECLIR - eCR Sink.....Sink	12



no-op Route Error Handler In

Filter

No-operation

On ECLIR - eCR Messages In route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - eCR Messages In Route Error Handler* when an unhandled error occurs anywhere in the route.

Destinations

Name	Page
js Set Error properties..... JavaScript (v.1)	49



no-op eICR Msgs To Parsing Route

Filter

No-operation

On ECLIR - eCR Messages In route

Notes

Purpose: This No-operation filter sends eICR messages to the *ECLIR - eICR Condition Checks* route.

Destinations

Name	Page
no-op eCR Msgs In..... No-operation	112



RR

Filter

No-operation

On ECLIR - eCR Messages In route

Notes

Purpose: This No-operation filter sends RR (Reportability Response) messages to the *js Extract Reportables From RR and pair eICR JavaScript Filter*.

Destinations

Name	Page
 js Extract Reportables From RR and pair eICR..... JavaScript (v.1)	42



js Set collectionID Property

Filter

JavaScript (v.1)

On ECLIR - eCR Messages In route

Notes

Purpose:

This JavaScript filter extracts the document unique ID from the eICR or the RR and saves it to the collectionID message property to be used for the Message Collector in the *js Extract Reportables From RR and pair eICR* JavaScript Filter.

The type of document is set to the docType message property as wither 'eicr' or 'rr' to be used for the conditional connectors output from this filter.

Message properties that are set in this filter are:

collectionID
docType
aimsSender
aimsSenderName

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

var docType = 'unknown CDA';
var fileName = '';
var collectionID = '';

if (next.getProperty('BaseFilename') != null) {
  fileName = next.getProperty('BaseFilename').toLowerCase();
}
else if (next.getProperty('ZipBaseFilename') != null) {
  fileName = next.getProperty('ZipBaseFilename').toLowerCase();
}
next.setProperty('rr_ORIGINAL_FILE_NAME', fileName);

// Determine document type and set property. Either eICR or RR.
if (next.getField('/ClinicalDocument/code/@code') == "55751-2") {
  docType = 'eicr';
}
else if (next.getField('/ClinicalDocument/code/@code') == "88085-6") {
```

Javascript

```

docType = 'rr';
}
next.setProperty('docType', docType);

//
=====
//_____ INITIAL VARIABLES (end
)
//
=====

//
=====
//_____ SET AIMSPLATFORM MESSAGE PROPERTIES (start
)
//
=====

var strAIMS_JSON = JSON.parse(next.getProperty('AIMSPlatformSender'));

next.setProperty('aimsSender', strAIMS_JSON.sender);
next.setProperty('aimsSenderName', strAIMS_JSON.senderName);

//
=====
//_____ SET AIMSPLATFORM MESSAGE PROPERTIES (end
)
//
=====

//
=====
//_____ SET collectionID PROPERTY (start
)
//
=====
// Set collectionID property for RR and extract condition codes.

if (docType == 'rr') {
    var eicrID_root = next.getField('/ClinicalDocument/component/structuredBody/component[section
/code/@code="88082-3"]/section/entry[act/code/@code="RR5"]/act/reference/externalDocument/id/
@root');
    if (eicrID_root != null) {
        collectionID = eicrID_root;
    }
    var eicrID_extension = next.getField('/ClinicalDocument/component/structuredBody/component[sec
tion/code/@code="88082-3"]/section/entry[act/code/@code="RR5"]/act/reference/externalDocument/
id/@extension');
    if (eicrID_root != null && eicrID_root.indexOf('&') != -1) {collectionID = eicrID_root;}
    else if (eicrID_extension != null) {
        collectionID += '&' + eicrID_extension;
    }
}

```

Javascript

```
next.setProperty('collectionID', collectionID);
next.indexOutputProperty('collectionID');

}

// Set collectionID property for eICR.
else if (docType == 'eicr') {
  //Document Id Root
  var documentID_root = next.getField('/ClinicalDocument/id/@root');
  if (documentID_root != null) {collectionID = documentID_root;}

  //Document Id Extention
  var documentID_extension = next.getField('/ClinicalDocument/id/@extension');
  if (documentID_extension != null) {collectionID += '&' + documentID_extension;}
  next.setProperty('collectionID', collectionID);
  next.indexOutputProperty('collectionID');
}

//
=====
//_____ SET collectionID PROPERTY (end
)
//
=====
```

Destinations

Name	Page
Output Condition: eICR	
docType = 'eicr' ignoring case	
 eICR.....	No-operation 42
Output Condition: RR	
docType = 'rr' ignoring case	
 RR.....	No-operation 38
Error Connector	Page
 ECLIR - eCR Sink.....	Sink 12



Error Queue

On ECLIR - eCR Messages In route

Filter

Configuration Properties

Error Message	Sent from error filter
---------------	------------------------



no-op REH to Sink

Filter

No-operation

On ECLIR - eCR Messages In route

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - eCR Messages In Route Error Handler* to the *ECLIR - eCR Route Error Handler Sink* after the errors have been stored into the *ECR_ERROR* table in the ECLIR database.

Destinations

Name	Page
ECLIR - eCR Route Error Handler Sink.....Sink	11



eICR

Filter

No-operation

On ECLIR - eCR Messages In route

Notes

Purpose: This No-operation filter sends eICR (Initial Case Report) messages to the *js Extract Reportables From RR and pair eICR* JavaScript Filter.

Destinations

Name	Page
js Extract Reportables From RR and pair eICR.....JavaScript (v.1)	42



js Extract Reportables From RR and pair eICR

Filter

JavaScript (v.1)

On ECLIR - eCR Messages In route

Notes

Purpose:

This JavaScript filter pairs eICR and RR documents in the Message Collector based on the *collectionID* message property set in the *js Set collectionID Property* JavaScript Filter. Since the RR document will no longer be persisted in the subsequent routes, data is parsed from the RR and stored in message properties to be used in the *ECLIR - eICR Message Parsing* route to store information from the RR in the ECLIR database tables.

Notes

If more than one triggered condition is present in the RR, the data is concatenated together as '<condition code>==<data value>', '<condition code>==<data value>', '<condition code>==<data value>', etc. and written to the relevant message property. In the *ECLIR - eICR Message Parsing* route, the message properties will be split and written to an XML output in the *js Parse eICR to XML for DB Inserts* JavaScript filter that allows the data to be stored in the ECLIR database accordingly.

A message property `rr_Multiple_Conditions` is also set to indicate if the split logic needs to be used on these properties.

Message properties that are set in this filter are:

```
rr_GLOBAL_UNIQUE_ID
rr_INITIATION_INDICATOR
rr_eRSD_VERSION_20
rr_eRSD_VERSION_29
rr_eRSD_VERSION_31
rr_CODE_DETERMINATION
rr_RULE_DETERMINATION
rr_TEXT_DETERMINATION
rr_LOCATION_RELEVANCE_CODE
rr_LOCATION_RELEVANCE_NAME
rr_ROUTING_ENTITY_NAME
rr_ROUTING_ENTITY_DESCRIPTION
rr_RULES_AUTHORIZING_AGENCY
rr_MANUALLY_TRIGGERED_FL
rr_Multiple_Conditions
conditionCode
```

Javascript

```
//
=====
//_____INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var rrXML = input[1];

//Relevant Reportable Condition Observation component in the Summary section of the Reporta
bility Response
var componentReportableConditionObservation = '/ClinicalDocument/component/structuredBody/com
ponent/section/entry/organizer/component/observation[code/@code = "75323-6" or code/@code =
"64572001" or code/translation/@code = "75323-6" or code/translation/@code = "64572001"]';

//
=====
//_____INITIAL VARIABLES (end
)
//
=====

//
=====
```

Javascript

```
// _____ SET PARSED RR FIELDS TO PROPERTIES (start
)
//
=====
// Store key fields from the RR into message properties to be written to the ECLIR database
later.

var manInitCode = rrXML.getField('/ClinicalDocument/component/structuredBody/component/section/
entry/act[templateId/@root = "2.16.840.1.113883.10.20.15.2.3.22"]/@code/@code');
var manInitDisplayName = rrXML.getField('/ClinicalDocument/component/structuredBody/component/
section/entry/act[templateId/@root = "2.16.840.1.113883.10.20.15.2.3.22"]/@code/@displayName');
var manInitCodeSystemName = rrXML.getField('/ClinicalDocument/component/structuredBody/com
ponent/section/entry/act[templateId/@root = "2.16.840.1.113883.10.20.15.2.3.22"]/@code/@code
SystemName');
var manInitCodeSystem = rrXML.getField('/ClinicalDocument/component/structuredBody/component/
section/entry/act[templateId/@root = "2.16.840.1.113883.10.20.15.2.3.22"]/@code/@codeSystem');
var rr_INITIATION_INDICATOR = manInitCode + '^' + manInitDisplayName + '^' + manInitCodeSystem
Name + '^' + manInitCodeSystem;
var rr_eRSD_VERSION_20 = rrXML.getField('/ClinicalDocument/component/structuredBody/component/
section/entry/act/code[@code="RRVS20"]/@displayName');
var rr_eRSD_VERSION_29 = rrXML.getField('/ClinicalDocument/component/structuredBody/component/
section/entry/act/entryRelationship/observation/value[@code="RRVS29"]/@displayName');
var rr_eRSD_VERSION_31 = rrXML.getField('/ClinicalDocument/component/structuredBody/component
/section/entry/act/entryRelationship/observation/entryRelationship/observation/code[@code
="RRVS31"]/@value/@displayName');

//-----eICR_REPORTABLE_RESPONSE table columns-----
next.setProperty('rr_GLOBAL_UNIQUE_ID', rrXML.getField('/ClinicalDocument/id/@root'));
next.setProperty('rr_INITIATION_INDICATOR', rr_INITIATION_INDICATOR);
next.setProperty('rr_eRSD_VERSION_20', rr_eRSD_VERSION_20);
next.setProperty('rr_eRSD_VERSION_29', rr_eRSD_VERSION_29);
next.setProperty('rr_eRSD_VERSION_31', rr_eRSD_VERSION_31);

var rr_Multiple_Conditions = false;
if (rrXML.getRepeatCount(componentReportableConditionObservation + '/value/@code') > 1) {
  rr_Multiple_Conditions = true;
}
var conditionCode = rrXML.getFieldAsList(componentReportableConditionObservation + '/value/
@code');

//-----eICR_DOCUMENT_TRIGGER table columns-----

if (rrXML.getField('/ClinicalDocument/component/structuredBody/component/section/entry/act/code
[@code="PHC1464"]/@code') != null) { var rr_MANUALLY_TRIGGERED_FL = '1'; }
else { var rr_MANUALLY_TRIGGERED_FL = '0'; }

var rr_CODE_DETERMINATION = '';
var rr_RULE_DETERMINATION = '';
var rr_TEXT_DETERMINATION = '';
var rr_LOCATION_RELEVANCE_CODE = '';
var rr_LOCATION_RELEVANCE_NAME = '';
var rr_ROUTING_ENTITY_NAME = '';
var rr_ROUTING_ENTITY_DESCRIPTION = '';
var rr_RULES_AUTHORING_AGENCY = '';
```

Javascript

```

for (i=0; i < rrXML.getRepeatCount(componentReportableConditionObservation); i++) {
    var strDelim = '^';
    if (i==0) { strDelim = ''; }
    rr_RULE_DETERMINATION += strDelim + conditionCode[i] + '==' + rrXML.getField(componentReportableConditionObservation + '[' + (i+1) + ']/entryRelationship/organizer/component[1]/observation/entryRelationship/observation/value');
    rr_TEXT_DETERMINATION += strDelim + conditionCode[i] + '==' + rrXML.getField(componentReportableConditionObservation + '[' + (i+1) + ']/entryRelationship/organizer/component/observation/entryRelationship/observation[code/@code = "RR3"]/value');
    rr_LOCATION_RELEVANCE_CODE += strDelim + conditionCode[i] + '==' + rrXML.getField(componentReportableConditionObservation + '[' + (i+1) + ']/entryRelationship/organizer[templateId/@root = "2.16.840.1.113883.10.20.15.2.3.13"]/code/@code');
    rr_LOCATION_RELEVANCE_NAME += strDelim + conditionCode[i] + '==' + rrXML.getField(componentReportableConditionObservation + '[' + (i+1) + ']/entryRelationship/organizer[templateId/@root = "2.16.840.1.113883.10.20.15.2.3.13"]/code/@displayName');
    rr_ROUTING_ENTITY_NAME += strDelim + conditionCode[i] + '==' + rrXML.getField(componentReportableConditionObservation + '[' + (i+1) + ']/entryRelationship/organizer/participant/participantRole[code/@code = "RR7"]/playingEntity/name');
    rr_ROUTING_ENTITY_DESCRIPTION += strDelim + conditionCode[i] + '==' + rrXML.getField(componentReportableConditionObservation + '[' + (i+1) + ']/entryRelationship/organizer/participant/participantRole[code/@code = "RR7"]/playingEntity/desc');
    rr_RULES_AUTHORIZING_AGENCY += strDelim + conditionCode[i] + '==' + rrXML.getField(componentReportableConditionObservation + '[' + (i+1) + ']/entryRelationship/organizer/participant/participantRole[code/@code = "RR12"]/playingEntity/name');
}
if (rr_Multiple_Conditions == true) {
    conditionCode = conditionCode.join('^');
}

rr_CODE_DETERMINATION = conditionCode;
next.setProperty('rr_CODE_DETERMINATION', rr_CODE_DETERMINATION);
next.setProperty('rr_RULE_DETERMINATION', rr_TEXT_DETERMINATION);
next.setProperty('rr_TEXT_DETERMINATION', rr_TEXT_DETERMINATION);
next.setProperty('rr_LOCATION_RELEVANCE_CODE', rr_LOCATION_RELEVANCE_CODE);
next.setProperty('rr_LOCATION_RELEVANCE_NAME', rr_LOCATION_RELEVANCE_NAME);
next.setProperty('rr_ROUTING_ENTITY_NAME', rr_ROUTING_ENTITY_NAME);
next.setProperty('rr_ROUTING_ENTITY_DESCRIPTION', rr_ROUTING_ENTITY_DESCRIPTION);
next.setProperty('rr_RULES_AUTHORIZING_AGENCY', rr_RULES_AUTHORIZING_AGENCY);
next.setProperty('rr_MANUALLY_TRIGGERED_FL', rr_MANUALLY_TRIGGERED_FL);

next.setProperty('rr_Multiple_Conditions', rr_Multiple_Conditions);
next.indexInputProperty('rr_Multiple_Conditions');
next.indexOutputProperty('rr_Multiple_Conditions');
next.setProperty('conditionCode', conditionCode);
next.indexInputProperty('conditionCode');
next.indexOutputProperty('conditionCode');

//
=====
// _____ SET PARSED RR FIELDS TO PROPERTIES (end)
)
//
=====

```

Message Collector: Input Collation

Collection Order

[ECLIR - eCR Messages In/ca1af5ac-b446-4f63-b90a-1051302c1856,ECLIR - eCR Messages In/6938acf1-c931-448b-bb7e-d20f9b6d03e4]

Message Collector: Input Collation	
Match Property	collectionID
Message Set Timeout	1
Timeout Units	Minutes
Timeout Destination	Error connector

Destinations

Name	Page
 no-op eICR Msgs To Parsing Route.....No-operation	38



db eCR Messages In Route Errors

Filter

Database LookUp

On ECLIR - eCR Messages In route

Notes

Purpose:

This Database Lookup filter inserts an error record into the eICR_ERROR table in the ECLIR database in the event an unhandled error is caught by the *ECLIR - eCR Messages In Route Error Handler*.

Based on the docType message property, the error is inserted into the eICR_ERROR table differently:

If the document is an RR document (docType = 'rr'), an eICR_DOCUMENT_PARSED_ID cannot be created, null is stored to that column and the message property collectionID is prepended with 'rr_' to be stored in the DOC_ID_CONCAT column.

If the document is an eICR document (docType = 'eicr'), the eICR_DOCUMENT_PARSED_ID column is populated in addition to the other fields.

Configuration Properties

Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	

Configuration Properties

Socket Timeout 0

Configuration File: Database LookUp 2.xml

```

<statement name="Check for existing eICR_DOCUMENT_ID" ide:type="executeSql" test="Null($eICR_DOCUMENT_PARSED_ID)" ide:summary="Before creating a new record in eICR_DOCUMENT_PARSED, check if one exists already and use that ID to write out errors." ide:allowCompoundStatements="true">
  <sql>SELECT ID AS eICR_DOCUMENT_PARSED_ID
  FROM eICR_DOCUMENT_PARSED
  WHERE DOCUMENT_OID = $collectionID</sql>
  <statement name="Insert Into eICR_ERROR table" ide:type="executeSql" test="NotNull(@eICR_DOCUMENT_PARSED_ID)" ide:summary="Add error record to the eICR_ERROR table after eICR_DOCUMENT_PARSED_ID is found." ide:allowCompoundStatements="true">
    <sql>INSERT INTO eICR_ERROR (
      eICR_DOCUMENT_PARSED_ID
    ,DOCUMENT_OID
    ,ERROR_CATEGORY
    ,ERROR_TYPE
    ,PROCESSING_STEP
    ,ERROR_MSG
    ,INSERT_DATE
    )
    VALUES (
      @eICR_DOCUMENT_PARSED_ID
    , $collectionID
    , $ERROR_CATEGORY
    , $ERROR_TYPE
    , $PROCESSING_STEP
    , $ERROR_MSG
    , SYSDATETIME ()
    )</sql>
  </statement>
  <statement name="Create eICR_DOCUMENT_PARSED_ID if possible" ide:type="executeSql" test="Null(@eICR_DOCUMENT_PARSED_ID)" ide:summary="If the docType is 'eicr' try to create an entry in the eICR_DOCUMENT_PARSED table." ide:allowCompoundStatements="true">
    <sql>INSERT INTO eICR_DOCUMENT_PARSED (
      eICRDQ_RECORD_ID
    ,DOCUMENT_OID
    ,PLANNED_ORDERS_FL
    ,DOCUMENT_SOURCE_BUCKET
    ,INSERT_DATE
    )
    VALUES (
      $eICRDQ_RECORD_ID
    , $collectionID
    , 0
    , $DOCUMENT_SOURCE_BUCKET
    , SYSDATETIME ()
    )</sql>
    <statement ide:type="compoundSqlStatement">
      <sql>SELECT \@\\@IDENTITY AS eICR_DOCUMENT_PARSED_ID</sql>
      <substituteColumn name="eICR_DOCUMENT_PARSED_ID">$eICR_DOCUMENT_PARSED_ID</substituteColumn>
    </statement>

```

Configuration File: Database LookUp 2.xml

```

    <statement name="Insert Into eICR_ERROR table" ide:type="executeSql" ide:summary="Add
error record to the eICR_ERROR table." ide:allowCompoundStatements="true">
    <sql>INSERT INTO eICR_ERROR (
eICR_DOCUMENT_PARSED_ID
,DOCUMENT_OID
,ERROR_CATEGORY
,ERROR_TYPE
,PROCESSING_STEP
,ERROR_MSG
,INSERT_DATE
)
VALUES (
    @eICR_DOCUMENT_PARSED_ID
    , $collectionID
    , $ERROR_CATEGORY
    , $ERROR_TYPE
    , $PROCESSING_STEP
    , $ERROR_MSG
    , SYSDATETIME()
) </sql>
    </statement>
    </statement>
    </statement>
    <statement name="Insert Into eICR_ERROR table" ide:type="executeSql" test="NotNull($eICR_DOC
UMENT_PARSED_ID)" ide:summary="Add error record to the eICR_ERROR table." ide:allowCompoundStat
ements="true">
    <sql>INSERT INTO eICR_ERROR (
eICR_DOCUMENT_PARSED_ID
,DOCUMENT_OID
,ERROR_CATEGORY
,ERROR_TYPE
,PROCESSING_STEP
,ERROR_MSG
,INSERT_DATE
)
VALUES (
    $eICR_DOCUMENT_PARSED_ID
    , $collectionID
    , $ERROR_CATEGORY
    , $ERROR_TYPE
    , $PROCESSING_STEP
    , $ERROR_MSG
    , SYSDATETIME()
) </sql>
    </statement>
</config>

```

Destinations

Name	Page
 no-op REH to Sink..... No-operation	42
Error Connector	Page
 Error Queue..... Error Queue	41



no-op eCR Msgs In

Filter

No-operation

On ECLIR - eCR Messages In route

Notes

Purpose: This No-operation filter receives eCR messages from the existing [redacted] routes.

Destinations

Name	Page
Output Condition: Zipped Files	
FullFilename contains 'zip' ignoring case	
Unzip File.....Zip / Unzip	37
Error Connector	Page
no-op Filter Messages.....No-operation	37



js Set Error properties

Filter

JavaScript (v.1)

On ECLIR - eCR Messages In route

Notes

Purpose: This JavaScript filter
Sets the property ERROR_CATEGORY with the type of error to be stored in the ERROR_CATEGORY column in the eICR_ERROR table. The category is set to 'FATAL' because the message stops processing.
Sets the property ERROR_TYPE with the type of error to be stored in the ERROR_TYPE column in the eICR_ERROR table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.
Sets the property PROCESSING_STEP with the Rhapsody route name and filter name to be stored in the PROCESSING_STEP column in the eICR_ERROR table.
Sets the property ERROR_MSG with the Rhapsody generated errors to be stored in the ERROR_MSG column in the eICR_ERROR table.
Sets the docType property to either 'rr' or 'eicr' to determine the document type. If this is an RR, the eICR_DOCUMENT_PARSED_ID cannot be created and the collectionID property is prepended with "rr_" to be written to the DOC_ID_CONCAT column for tracking purposes and to indicate that the failed message was an RR.
Sets the property collectionID to be stored in the DOC_ID_CONCAT column in the eICR_ERROR table.

Javascript

```
//  
===== INITIAL VARIABLES (start  
)
```

Javascript

```
//
=====

var next = output.append(input[0]);

//
=====
//_____ INITIAL VARIABLES (end
)_____
//
=====

//
=====
//_____ SET MESSAGE PROPERTIES FOR DB INSERT (start
)_____
//
=====

// Determine document type and set property. Either eICR or RR.
var fileName = '';
var collectionID = '';

if (next.getProperty('BaseFilename') != null) {
    fileName = next.getProperty('BaseFilename').toLowerCase();
}
else if (next.getProperty('ZipBaseFilename') != null) {
    fileName = next.getProperty('ZipBaseFilename').toLowerCase();
}

// Set collectionID property for RR.
if (fileName.indexOf('_rr') != -1) {
    next.setProperty('docType', 'rr');

    var eicrID_root = next.getField('/ClinicalDocument/component/structuredBody/component[section/code/@code="88082-3"]/section/entry[act/code/@code="RR5"]/act/reference/externalDocument/id/@root');
    if (eicrID_root != null) {
        collectionID = eicrID_root;
    }
    var eicrID_extension = next.getField('/ClinicalDocument/component/structuredBody/component[section/code/@code="88082-3"]/section/entry[act/code/@code="RR5"]/act/reference/externalDocument/id/@extension');
    if (eicrID_root != null && eicrID_root.indexOf('&') != -1) {collectionID = eicrID_root;}
    else if (eicrID_extension != null) {
        collectionID += '&' + eicrID_extension;
    }

    next.setProperty('collectionID', 'RR_' + collectionID);
    next.indexOutputProperty('collectionID');
}

// Set collectionID property for eICR.
else if (fileName.indexOf('_eicr') != -1) {
```

Javascript

```

next.setProperty('docType', 'eicr');

//Document Id Root
var documentID_root = next.getField('/ClinicalDocument/id/@root');
if (documentID_root != null) {collectionID = documentID_root;}

//Document Id Extention
var documentID_extension = next.getField('/ClinicalDocument/id/@extension');
if (documentID_extension != null) {collectionID += '&' + documentID_extension;}
next.setProperty('collectionID', collectionID);
next.indexOutputProperty('collectionID');
}

next.setProperty('ERROR_CATEGORY', 'FATAL');
next.setProperty('ERROR_TYPE', 'Rhapsody');
next.setProperty('PROCESSING_STEP', 'Parsing: ECLIR - eICR Message Parsing (Route) ->' + next
.getProperty('rhapsody:ErrorFilterName') + ' (Filter)');
next.setProperty('ERROR_MSG', 'Internal Rhapsody Route Error. Message processing could not con
tinue: ' + next.getErrors());

//Set property for 'DOCUMENT_SOURCE_BUCKET' column based on the input comm point.
var DOCUMENT_SOURCE_BUCKET = next.getProperty('rhapsody:InputCommunicationPoint');
if (DOCUMENT_SOURCE_BUCKET == 'AIMS Platform') {
  DOCUMENT_SOURCE_BUCKET = '■_PROD';
}
else {
  DOCUMENT_SOURCE_BUCKET = '■_TEST';
}
next.setProperty('DOCUMENT_SOURCE_BUCKET', DOCUMENT_SOURCE_BUCKET);

//
=====
//_____ SET MESSAGE PROPERTIES FOR DB INSERT (end
)_____
//
=====

```

Destinations

Name	Page
 db eCR Messages In Route Errors..... Database LookUp	46



ECLIR - eCR Polling

Folder

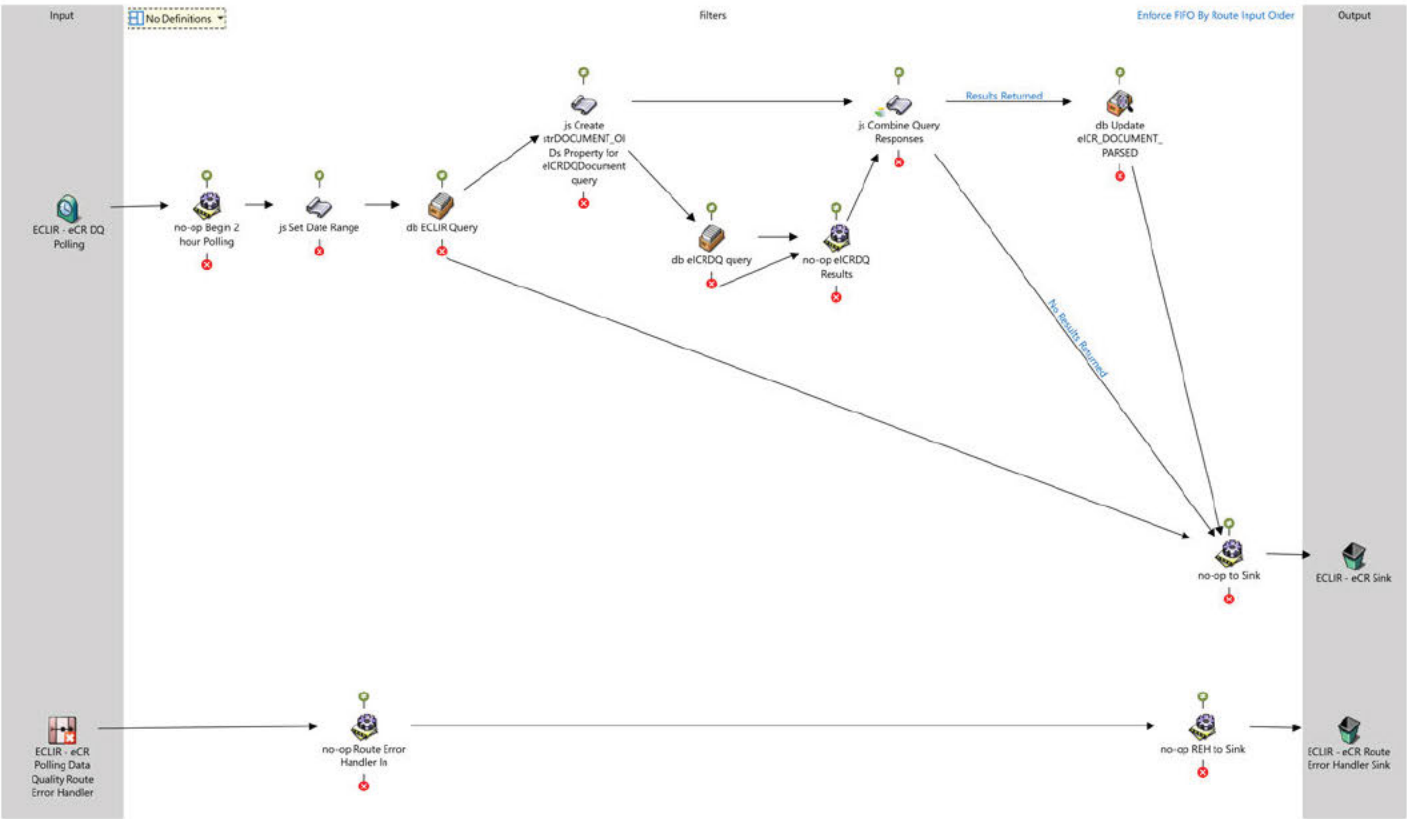
Full Path /ECLIR/Route__ECLIR/eCR/ECLIR - eCR Polling

Summary	
Routes (3)	
Folder Contents	Page
ECLIR - eCR Polling Data Quality.....Route	53
ECLIR - eCR Polling NBS First Pass.....Route	65
ECLIR - eCR Polling NBS Patient.....Route	87



ECLIR - eCR Polling Data Quality

Route



Notes

Purpose: This route triggers every 2 hours starting at 5 minutes past the hour to query the eCR Data Quality table eICRDQDocument table for the record_ID and saves it to the ECLIR table eICR_DOCUMENT_PARSED in order to link eCR documents together.

Inputs	Page
ECLIR - eCR DQ Polling..... Timer	16
Outputs	Page
ECLIR - eCR Sink..... Sink	12
ECLIR - eCR Route Error Handler Sink..... Sink	11

Filters

Name	Page
db ECLIR Query..... Database Message Extraction	61
db eICRDQ query..... Database Message Extraction	56
db Update eICR_DOCUMENT_PARSED..... Database LookUp	55

Name	Page
⚡ js Combine Query Responses..... JavaScript (v.1)	62
⚡ js Create strDOCUMENT_OIDs Property for eICRDQDocument query..... JavaScript (v.1)	59
⚡ js Set Date Range..... JavaScript (v.1)	57
🛡 no-op Begin 2 hour Polling..... No-operation	55
🛡 no-op eICRDQ Results..... No-operation	64
🛡 no-op REH to Sink..... No-operation	64
🛡 no-op Route Error Handler In..... No-operation	59
🛡 no-op to Sink..... No-operation	63

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



no-op Begin 2 hour Polling

Filter

No-operation

On ECLIR - eCR Polling Data Quality route

Notes

Purpose: This No-operation filter receives a trigger message every 2 hours from the *ECLIR - eCR DQ Polling* Timer to begin the polling process for eCR.

Destinations

Name

Page

js Set Date Range..... JavaScript (v.1) [57](#)



db Update eICR_DOCUMENT_PARSED

Filter

Database LookUp

On ECLIR - eCR Polling Data Quality route

Notes

Purpose: This Database Lookup filter updates the eICR_DOCUMENT_PARSED table with an eCR Data Quality identifier returned from the previous filter.

Configuration Properties

Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	

Configuration Properties	
Socket Timeout	0
Configuration File: db Update eICR_DOCUMENT_PARSED.xml	
<pre> <statement name="Update eICR_DOCUMENT_PARSED.eICRDQ_RECORD_ID" ide:type="executeSql" ite rate="eICR_DOCUMENT_PARSED_RECORD" test="NotNull(`/message/eICR_DOCUMENT_PARSED_RECORD/eICRDQ _RECORD_ID`)" ide:summary="Update eICR_DOCUMENT_PARSED eICRDQ_RECORD_ID column from the Data Quality routes." ide:allowCompoundStatements="true"> <sql>UPDATE eICR_DOCUMENT_PARSED SET eICRDQ_RECORD_ID = `/message/eICR_DOCUMENT_PARSED_RECORD/eICRDQ_RECORD_ID` WHERE DOCUMENT_OID = `/message/eICR_DOCUMENT_PARSED_RECORD/DOCUMENT_OID`</sql> </statement> </config> </pre>	

Destinations

Name	Page
 no-op to Sink..... No-operation	63



db eICRDQ query

Database Message Extraction

On ECLIR - eCR Polling Data Quality route

Filter

Notes
Purpose: This Database Message Extraction filter returns an XML list of records from the eICRDQDocument table in the eCR Data Quality database.

Configuration Properties		
Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	Plain text	

Configuration Properties	
Definition	
Query Timeout	
Deadlock Retry Attempts	5
Deadlock Retry Delay	100
Socket Timeout	0
Configuration File: Database Message Extraction.xml	
<pre> <statement name="eICRDQ_RECORD" ide:type="executeSql" ide:summary="Retrieve the record_ID from the eICRDQDocument for each matching doc_ID_concat from the previous query." ide:allow CompoundStatements="true"> <sql>SELECT eICRDQDocument.record_ID AS eICRDQ_RECORD_ID, eICRDQDocument.doc_ID_concat AS DOCUMENT_OID FROM eICRDQDocument WHERE eICRDQDocument.doc_ID_concat IN (SELECT * FROM STRING_SPLIT(\$strDOCUMENT_OIDs, ' '))</sql> </statement> </config> </pre>	

Destinations

Name	Page
 no-op eICRDQ Results.....No-operation	64
Error Connector	Page
 no-op eICRDQ Results.....No-operation	64



js Set Date Range

JavaScript (v.1)

On ECLIR - eCR Polling Data Quality route

Filter

Notes

Purpose: This JavaScript filter sets the message property DateRange to be used in the Database Lookup filter *db ECLIR Query* to allow the query to only select records from the last 30 days for querying.

Javascript

```

//
=====
//_____ INITIAL VARIABLES (start
)_____
//
=====

var next = output.append(input[0]);

```

Javascript

```

var pollingDR = getVariable('DR_ECR_DQ');
next.text = 'eICR Data Quality Polling';

//
=====
//_____ INITIAL VARIABLES (end
)_____
//
=====

//
=====
//_____ FUNCTIONS (start
)_____
//
=====

//FUNCTION: fnPadZeros
//--- Function to pad a number with leading zero.
function fnPadZeros(num, size) {
    num = num.toString();
    while (num.length < size) num = "0" + num;
    return num;
}

//
=====
//_____ FUNCTIONS (end
)_____
//
=====

//
=====
//_____ SET MESSAGE PROPERTY DateRange (start
)_____
//
=====

var dateRange = new Date();
dateRange.setDate(dateRange.getDate() - pollingDR);
var fieldYear = dateRange.getFullYear();
var fieldMonth = fnPadZeros(dateRange.getMonth() + 1, 2);
var fieldDay = fnPadZeros(dateRange.getDate(), 2);
var fieldHours = fnPadZeros(dateRange.getHours(), 2);
var fieldMinutes = fnPadZeros(dateRange.getMinutes(), 2);
var fieldSeconds = fnPadZeros(dateRange.getSeconds(), 2);

var formatteddateRange = fieldYear+'-'+fieldMonth+'-'+fieldDay+' '+fieldHours+':'+fieldMinutes
+':'+fieldSeconds;

next.setProperty('DateRange', formatteddateRange);

```

Javascript

```
next.setProperty('pollingDR', pollingDR) ;

//
=====
// _____ SET MESSAGE PROPERTY DateRange (end
)
//
=====
```

Destinations

Name	Page
 db ECLIR Query..... Database Message Extraction	61



no-op Route Error Handler In

Filter

No-operation
On ECLIR - eCR Polling Data Quality route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - eCR Polling Data Quality Route Error Handler* when an unhandled error occurs anywhere in the route. These errors cannot be recorded in the `eICR_ERROR` table because there will be multiple messages and no way to know which message caused the error.

Destinations

Name	Page
 no-op REH to Sink..... No-operation	64



js Create strDOCUMENT_OIDs Property
for eICRDQDocument query

Filter

JavaScript (v.1)
On ECLIR - eCR Polling Data Quality route

Notes

Purpose: This JavaScript filter sets the message property `strDOCUMENT_OIDs` to be used in the Database Lookup filter *db eICRDQ Query* to query based on the `DOCUMENT_OID` of the documents.

```

=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var strDOCUMENT_OIDs = '';

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTY strDOCUMENT_OIDs (start
)
//
=====

var arrDOCUMENT_OIDs = next.getFieldAsList('/message/eICR_DOCUMENT_PARSED_RECORD/DOCUMENT
_OID');
for (var i=0; i < arrDOCUMENT_OIDs.length; i++) {
  if (strDOCUMENT_OIDs != '') {
    strDOCUMENT_OIDs += '|';
  }
  if (arrDOCUMENT_OIDs[i].indexOf('&') == -1) {
    strDOCUMENT_OIDs += arrDOCUMENT_OIDs[i] + '&';
  }
  else {
    strDOCUMENT_OIDs += arrDOCUMENT_OIDs[i];
  }
}

next.setProperty('strDOCUMENT_OIDs', strDOCUMENT_OIDs);

//
=====
// _____ SET MESSAGE PROPERTY strDOCUMENT_OIDs (end
)
//
=====

```

Destinations

Name	Page
 db eICRDQ query..... Database Message Extraction	56
 js Combine Query Responses..... JavaScript (v.1)	62



db ECLIR Query

Database Message Extraction

On ECLIR - eCR Polling Data Quality route

Filter

Notes

Purpose: This Database Message Extraction filter returns an XML list of records from the eICR_DOCUMENT_PARSED table that do not yet have the eCR Data Quality identifier recorded.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	[REDACTED]
Port	[REDACTED]	[REDACTED]
Database Name	[REDACTED]	[REDACTED]
Username	[REDACTED]	[REDACTED]
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File: Database Message Extraction.xml

```

[REDACTED]
[REDACTED]
[REDACTED]
<statement name="eICR_DOCUMENT_PARSED_RECORD" ide:type="executeSql" ide:summary="Queries the
eICR_DOCUMENT_PARSED table to records with missing eICRDQ_RECORD_ID." ide:allowCompoundStat
ements="true">
  <sql>SELECT TOP(500) eICR_DOCUMENT_PARSED.DOCUMENT_OID AS DOCUMENT_OID
FROM eICR_DOCUMENT_PARSED
WHERE eICR_DOCUMENT_PARSED.eICRDQ_RECORD_ID IS NULL AND
eICR_DOCUMENT_PARSED.INSERT_DATE >= $DateRange</sql>
</statement>
</config>

```

Destinations

Name	Page
 js Create strDOCUMENT_OIDs Property for eICRDQDocument query.....JavaScript (v.1)	59
Error Connector	Page
 no-op to Sink.....No-operation	63



js Combine Query Responses

Filter

JavaScript (v.1)

On ECLIR - eCR Polling Data Quality route

Notes

Purpose: This JavaScript filter combines the results from the Database Message Extraction filters *db ECLIR Query* and *db eICRDQ Query* into an XML file that will be used in the Database Lookup filter *db Update eICR_DOCUMENT_PARSED* to insert any Data Quality record_id's received.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var dbeICRDQ_Results = input[0];
var dbECLIR_Results = output.append(input[1]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ COMBINE QUERY RESPONSES FROM ECLIR AND eICRDQ (start
)
//
=====

if (dbeICRDQ_Results.getRepeatCount('/message/eICRDQ_RECORD') > 0) {
    dbECLIR_Results.setProperty('NoResults', false);
}
```

Javascript

```

for (var i=1; i <= dbECLIR_Results.getRepeatCount('/message/eICR_DOCUMENT_PARSED_RECORD'); i++) {
    var pathECLIR_results = '/message/eICR_DOCUMENT_PARSED_RECORD[' + i + ']';
    var DOCUMENT_OID = dbECLIR_Results.getField(pathECLIR_results + '/DOCUMENT_OID');
    if (DOCUMENT_OID.indexOf('&') == -1) { DOCUMENT_OID += '&'; }
    var eICRDQ_RECORD_ID = dbEICRDQ_Results.getField('/message/eICRDQ_RECORD[DOCUMENT_OID="' + DOCUMENT_OID + '"]/eICRDQ_RECORD_ID');
    if (eICRDQ_RECORD_ID != null) {
        if (eICRDQ_RECORD_ID.indexOf(',') != -1) { eICRDQ_RECORD_ID = eICRDQ_RECORD_ID.substr(eICRDQ_RECORD_ID.lastIndexOf(',')+1); }
        dbECLIR_Results.setField(pathECLIR_results + '/eICRDQ_RECORD_ID', eICRDQ_RECORD_ID);
    }
}
//If no results are returned from the eICRDQ query, send to Sink.
else {
    dbECLIR_Results.setProperty('NoResults', true);
}

//
=====
//_____ COMBINE QUERY RESPONSES FROM ECLIR AND eICRDQ (end)
)
//
=====

```

Message Collector: Input Collation

Collection Order	[ECLIR - eCR Polling Data Quality/fcd0de46-60d2-45d8-b1db-1b31952b4a51 ,ECLIR - eCR Polling Data Quality/5f878e08-cc28-4ef6-9423-dc5d7eeca612]
Match Property	InputTime
Message Set Timeout	10
Timeout Units	Minutes
Timeout Destination	Error connector

Destinations

Name	Page
Output Condition: Results Returned	
NoResults = 'false'	
 db Update eICR_DOCUMENT_PARSED..... Database Lookup	55
Output Condition: No Results Returned	
NoResults = 'true'	
 no-op to Sink..... No-operation	63



no-op to Sink
No-operation

Filter

On ECLIR - eCR Polling Data Quality route

Notes

Purpose: This No-operation filter sends all messages to the *ECLIR - eCR Sink* after queries are completed.

Destinations

Name	Page
 ECLIR - eCR Sink.....Sink	12

**no-op REH to Sink**

Filter

No-operation
On ECLIR - eCR Polling Data Quality route

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - eCR Polling Data Quality Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink*.

Destinations

Name	Page
 ECLIR - eCR Route Error Handler Sink.....Sink	11

**no-op eICRDQ Results**

Filter

No-operation
On ECLIR - eCR Polling Data Quality route

Notes

Purpose: This No-operation filter receives query results from *db eICRDQ Query* for both results found and empty results.

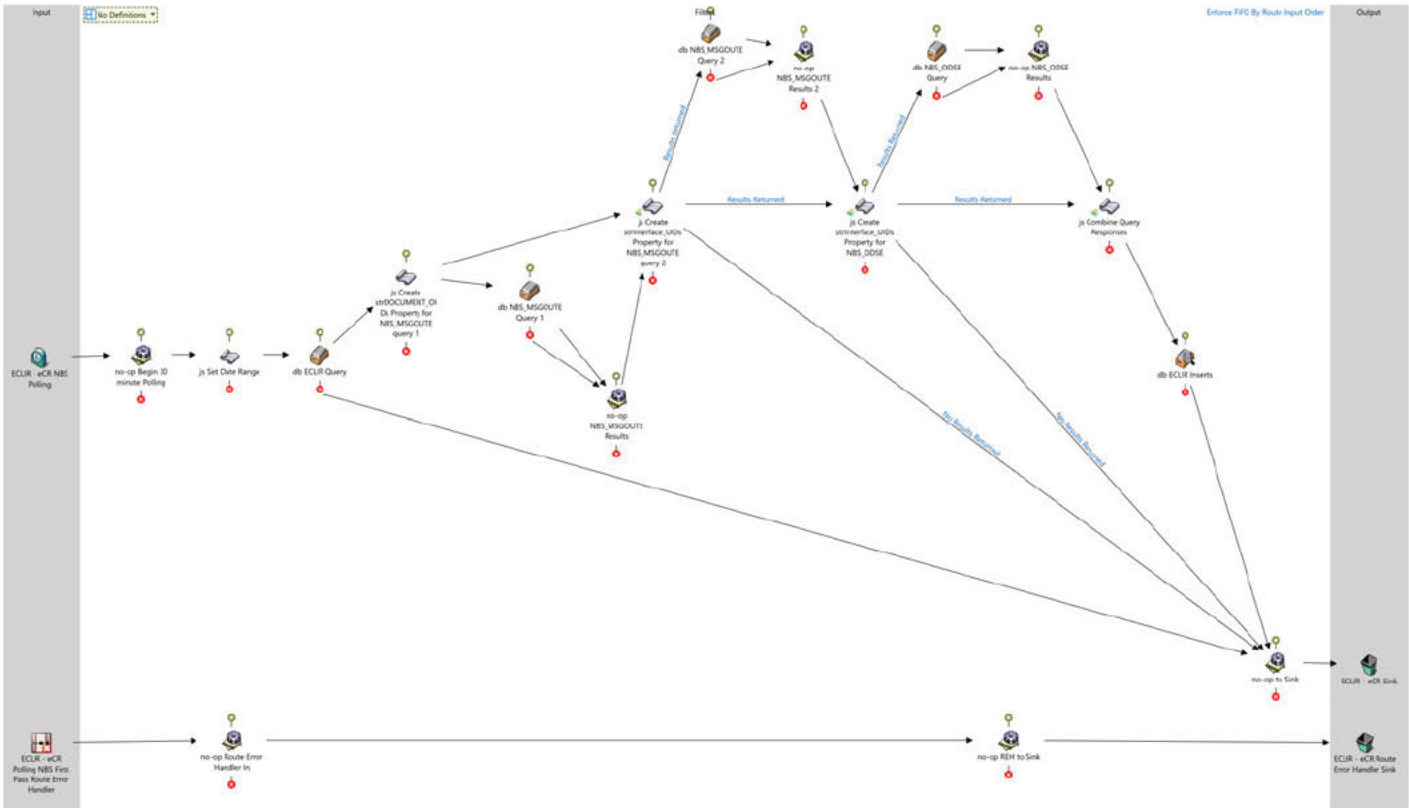
Destinations

Name	Page
 js Combine Query Responses.....JavaScript (v.1)	62



ECLIR - eCR Polling NBS First Pass

Route



Notes

Purpose: This route triggers every 30 minutes starting at 10 minutes past the hour to query NBS tables for key identifiers and stores them to the ECLIR database to allow messages to be linked together throughout the life of the message.

Inputs	Page
ECLIR - eCR NBS Polling..... Timer	18
Outputs	Page
ECLIR - eCR Sink.....Sink	12
ECLIR - eCR Route Error Handler Sink.....Sink	11

Filters

Name	Page
db ECLIR Inserts..... Database LookUp	72
db ECLIR Query..... Database Message Extraction	78
db NBS_MSGOUTE Query 1..... Database Message Extraction	68
db NBS_MSGOUTE Query 2..... Database Message Extraction	67

Name	Page
 db NBS_ODSE Query..... Database Message Extraction	82
 js Combine Query Responses..... JavaScript (v.1)	84
 js Create strDOCUMENT_OIDs Property for NBS_MSGOUTE query 1..... JavaScript (v.1)	76
 js Create strInterface_UIDs Property for NBS_MSGOUTE query 2..... JavaScript (v.1)	80
 js Create strInterface_UIDs Property for NBS_ODSE..... JavaScript (v.1)	74
 js Set Date Range..... JavaScript (v.1)	70
 no-op Begin 30 minute Polling..... No-operation	67
 no-op NBS_MSGOUTE Results..... No-operation	77
 no-op NBS_MSGOUTE Results 2..... No-operation	79
 no-op NBS_ODSE Results..... No-operation	79
 no-op REH to Sink..... No-operation	82
 no-op Route Error Handler In..... No-operation	72
 no-op to Sink..... No-operation	84

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



no-op Begin 30 minute Polling

[Filter](#)

No-operation

On ECLIR - eCR Polling NBS First Pass route

Notes

Purpose: This No-operation filter receives a trigger message every 30 minutes from the *ECLIR - eCR NBS Polling* Timer to begin the polling process for eCR.

Destinations

Name

Page

js Set Date Range..... JavaScript (v.1) [70](#)



db NBS_MSGOUTE Query 2

[Filter](#)

Database Message Extraction

On ECLIR - eCR Polling NBS First Pass route

Notes

Purpose: This Database Message Extraction filter returns an XML list of records with key identifiers from the NBS_MSG_OUTE.nbs_interface table. This query is a continuation from the previous filter to minimize the impact to the database.

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	[REDACTED]
Port	[REDACTED]	[REDACTED]
Database Name	[REDACTED]	[REDACTED]
Username	[REDACTED]	[REDACTED]
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	Plain text	
Definition		
Query Timeout		

Configuration Properties	
Deadlock Retry Attempts	5
Deadlock Retry Delay	100
Socket Timeout	0
Configuration File: Database Message Extraction.xml	
<pre> <statement name="NBS_INTERFACE_Record_2" ide:type="executeSql" ide:summary="Query NBS_MSGOUTE .NBS_INTERFACE table for key identifiers." ide:allowCompoundStatements="true"> <sql>SELECT ifc.nbs_interface_uid AS NBS_Interface_UID, ifc.record_status_cd AS INTERFACE_IMPORT_STATUS, ifc.record_status_time AS INTERFACE_IMPORT_STATUS_TIME, ifc.add_time AS INTERFACE_ADDED_TIME, ifc.doc_type_cd AS INTERFACE_DOC_TYPE, ifc.system_nm AS INTERFACE_SYSTEM_NAME, ifc.original_doc_type_cd AS INTERFACE_ORIG_DOC_TYPE, con.MSG_CONTAINER_UID FROM [nbs_msgoute].[dbo].[NBS_interface] ifc LEFT OUTER JOIN (SELECT ROW_NUMBER() OVER (PARTITION BY NBS_INTERFACE_UID ORDER BY MSG_CONTAINER_UID DESC) AS rn, MSG_CONTAINER_UID, NBS_INTERFACE_UID FROM [nbs_msgoute].[dbo].[MSG_CONTAINER]) con ON ifc.nbs_interface_uid = con.NBS_INTERFACE_UID AND con.rn = 1 WHERE ifc.doc_type_cd = 'PHC236' AND ifc.record_status_cd IN ('Success', 'Failure') AND DATEDIFF(dd,ifc.record_status_time, GETDATE()) between 0 and \$pollingDR AND ifc.nbs_interface_uid IN (SELECT * FROM STRING_SPLIT(\$strInterface_UIDs, ' '))</sql> </statement> </config> </pre>	

Destinations

Name	Page
 no-op NBS_MSGOUTE Results 2..... No-operation	79
Error Connector	Page
 no-op NBS_MSGOUTE Results 2..... No-operation	79



db NBS_MSGOUTE Query 1

Database Message Extraction

On ECLIR - eCR Polling NBS First Pass route

Filter

Notes

Purpose: This Database Message Extraction filter returns an XML list of records with key identifiers from the NBS_MSG_OUTE.nbs_interface table based on the query response from the previous filter *db ECLIR Query*. Since this query is process intense, it is ran separately from the subsequent query to the NBS_MSG_OUTE.nbs_interface table to avoid complexities.

Notes

(12/26/2024) LP Created filter

Configuration Properties

Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED] [REDACTED]	
Port	[REDACTED] [REDACTED]	
Database Name	[REDACTED] [REDACTED]	
Username	[REDACTED] [REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	

Configuration File: Database Message Extraction.xml

```

<statement name="NBS_Interface_Record" ide:type="executeSql" ide:summary="Query NBS_interface
table in NBS_MSGOUTE to retrieve the original document id from a commented out line." ide
:allowCompoundStatements="true">
  <sql>SELECT i.nbs_interface_uid as INTERFACE_UID,
    SUBSTRING(i.doc_elem, PATINDEX('%root=%', i.doc_elem)+6,
      CHARINDEX(' ', i.doc_elem, PATINDEX('%root=%', i.doc_elem)+7) - (PAT
INDEX('%root=%', i.doc_elem)+6))
    as DOC_ROOT,
    CASE WHEN i.doc_elem like '%extension=%'
      THEN SUBSTRING(i.doc_elem, PATINDEX('%extension=%', i.doc_elem)+11,
        CHARINDEX(' ', i.doc_elem, PATINDEX('%extension=%', i.doc_elem)+12) - (PATINDEX('%ext
ension=%', i.doc_elem)+11))
      END as DOC_EXT,
    new_doc_id as NBS_GENERATED_ID
FROM
  (SELECT ifc.nbs_interface_uid,
    SUBSTRING(
      ifc.original_payload,
      CHARINDEX('<id', ifc.original_payload, PATINDEX('%<!--<id%', ifc
.original_payload))+4,
      (PATINDEX('%>-->%', ifc.original_payload) - CHARINDEX('<id', ifc
.original_payload, PATINDEX('%<!--<id%', ifc.original_payload)) - 4) as doc_elem,
      SUBSTRING(
        ifc.original_payload,
        CHARINDEX('root=', ifc.original_payload, PATINDEX('%[^-]<id root=%',
ifc.original_payload))+6,

```